INFORMATION SECURITY MIDTERM THEORY PROJECT • TOPIC 13

Security in Client-Server Android Applications

Android Permissions, Intent Security, and Local Data Protection

ACADEMIC DETAILS

Course: Introduction to Information Security

Instructor: Huynh Ngoc Tu, Ph.D

Class ID: N05 | Group ID: G02

PROJECT TEAM (GROUP G02)

Ngô Lâm Tiến (Leader) | **ID:** 524H0195

Nguyễn Công Thành | **ID:** 524H0191

Võ Huỳnh Duy Tân | **ID:** 524H0187

THREAT LANDSCAPE & ATTACK SURFACE

Why Android Security Matters

The Multi-Layered Attack Surface of Mobile Client-Server Systems

End User
Device Owner
→
Android Client
Application Logic
→
Storage / IPC
SQLite & Intents
→
HTTPS / Transit
Network Channel
→
Backend Server
Cloud APIs & DB

1. Permissions Boundary

Gates sensitive device hardware, location, contacts, and privileged system APIs.

Vulnerability: Over-privilege, requesting unnecessary dangerous permissions, and failing to handle runtime revocations.

Governs Device Capabilities

2. IPC & Intent Boundary

Governs message passing between distinct application processes and components.

Vulnerability: Exported components blindly trusting unvalidated Intent extras and forwarding nested payloads.

Mediates Inter-App Trust

3. Storage-at-Rest Boundary

Isolates private SQLite databases, cached tokens, and preferences via Linux UIDs.

Vulnerability: Plaintext data exposed via Android backups, rooted devices, or exported ContentProviders.

Protects Local Assets

4. Network Transit Boundary

Secures client-server REST API payloads and authentication headers over HTTPS.

Vulnerability: Man-in-the-Middle (MITM) traffic decryption if user-installed root CAs are trusted without pinning.

Secures Remote Transit

KEY TAKEAWAY: Android security is not a single perimeter — it requires defense-in-depth across all four boundaries.

SYSTEM ARCHITECTURE

Android Security Foundation

Kernel-Level Sandboxing & Component-Based Isolation

Linux Application Sandbox

Per-App Linux UID: Unique immutable UID (e.g. `u0_a142`) assigned at install time.

Process Isolation: Dedicated Linux process running an independent ART virtual machine.

Kernel File Permissions: Private directories (`/data/data/<pkg>`) protected with strict `0700` DAC access.

Primary OS Boundary: Prevents ordinary third-party apps from reading another app's memory or files.

Core Principle: Multi-tenant Linux OS model applied to mobile apps.

The 4 Core Components

Activity: Interactive visual screens and UI workflows.

Service: Background worker performing long-running tasks without UI (sync, audio).

BroadcastReceiver: Asynchronous listener reacting to system-wide or application broadcasts.

ContentProvider: Structured data management interface abstracting SQLite access for IPC.

Every component has distinct lifecycle and IPC semantics.

AndroidManifest.xml Authority

Authoritative Contract: Root XML file declaring all app components, permissions, and security policies.

Component Exposure: The `android:exported` attribute dictates public accessibility to external apps.

Permission Binding: Enforces required capabilities (`<uses-permission>`) and signature security gates.

Central Attack Surface: A single misconfiguration here can compromise the entire sandbox boundary.

Single authoritative security contract parsed at install time.

ACCESS CONTROL & GOVERNANCE

Permission Model & AndroidManifest.xml

Granular Access Governance & The Principle of Least Privilege

Normal (Install-Time)

LOW RISK

Minimal risk (`INTERNET`, `ACCESS_NETWORK_STATE`). Auto-granted upon installation without user prompt.

Dangerous (Runtime)

USER PROMPT

Accesses private personal data (`CAMERA`, `READ_CONTACTS`, `LOCATION`). Requires explicit runtime user consent.

Signature

SAME CERT ONLY

Highest security. Granted automatically ONLY if requesting app is signed by the exact same developer certificate.

Special / Privileged

SYSTEM SETTINGS

Advanced system capabilities (`SYSTEM_ALERT_WINDOW`). Requires manual user authorization in system settings.

The android:exported Gateway

exported="false": Component is not available to other applications through normal external component launching.

exported="true": Allows the component to be reached by other applications, subject to any additional permission or access-control requirements.

Android 12+ Explicit Mandate: Any Activity, Service, or Receiver with intent filters MUST declare `android:exported` explicitly.

Key Rule: *Exported does not automatically mean vulnerable.* Vulnerability occurs when externally reachable endpoints lack caller authorization or input validation.

```
<!— Secure Component Declaration: Closed by default —>
<activity
    android:name=".InternalAdminActivity"
    android:exported="false" />
```

Principle of Least Privilege:
Keep components non-exported unless public access is strictly required, and protect exported components with appropriate permissions.

INTER-PROCESS COMMUNICATION

Android Intents & IPC Mechanics

Message-Passing Channels Crossing Security Trust Boundaries

Explicit Intent (Targeted Component)

Direct Binding: Specifies the exact target class (e.g. `Intent(context, TargetActivity::class.java)`).

Commonly used for internal navigation; can also target known components in another application.

Interception Reduction: Avoids implicit component resolution, significantly reducing intent-interception risk.

Implicit Intent (Dynamic Resolution)

Dynamic Resolution: Specifies Action (e.g. `ACTION_VIEW`) and Data URI without naming a target class.

Android Package Manager matches intent against declared `<intent-filter>` tags across all apps.

Security Risk: Unintended third-party applications can register matching filters to intercept or spoof data.

Intent Object Structure

Action

e.g. `ACTION_SEND`, `ACTION_VIEW`. Declares requested operation.

Data & Type (URI)

e.g. `content://`, `https://`. Specifies data target.

Extras Bundle (Parcelable)

Key-value parameters. Can carry serialized **Nested Intents**!

CRITICAL SECURITY PRINCIPLE: When Intent data crosses an application boundary, treat externally controlled fields as untrusted input requiring strict validation.

VULNERABILITY MECHANICS

Attack #1: Intent Injection & Redirection

Confused Deputy Exploitation via Unvalidated Intent Forwarding

1. Attacker App
Crafts Intent containing nested target Intent extra

→

2. Trust Boundary
Dispatched to exported Victim Proxy Activity

→

3. Vulnerability Point
Proxy extracts nested Intent without validation

→

4. Execution
startActivity(next) runs internal private activity!

```
// VULNERABLE PROXY COMPONENT (Exported Activity)
class ProxyActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // 1. Attacker controls 'next_intent' in Extras
        val next = intent.getParcelableExtra<Intent>("next_intent")

        // 2. VULNERABILITY: Blind forwarding!
        // Launches unexported activity under Victim's UID
        startActivity(next)
    }
}
```

Exploitation Mechanics:
The victim application launches attacker-controlled Intent data without sufficiently validating the destination, allowing outside callers to indirectly execute unexported components (`exported="false"`).

Technical Root Causes & Mechanics

Confused Deputy Flaw: The victim app acts as a privileged proxy, executing internal actions on behalf of an unprivileged attacker.

Access Control Bypass: Private internal activities (`exported="false"`) cannot be reached directly, but CAN be invoked internally by the victim app itself.

Missing Destination Validation: The proxy fails to inspect `next.component`, verify destination package, or enforce an allowlist.

Root Cause: The security failure occurs when externally controlled IPC is trusted without sufficient authorization, destination validation, or input sanitization.

Impact: Complete bypass of component access boundaries

DATA AT REST VULNERABILITIES

Attack #2: Sensitive Data in Local Storage

Plaintext SQLite Databases & Secondary Exposure Vectors

SQLite: user_session.db

PLAINTEXT AT REST

- `id: 10482`
- `user_email: student@tdtu.edu.vn`
- `auth_token: eyJhbGciOiJIUzI1Ni ... (Plaintext JWT)`
- `api_secret: sec_live_98234ab8 ... (Plaintext Key)`

Real-World Secondary Exposure Vectors

Exported ContentProvider: SQL injection or traversal leaks raw DB files.

Backup Exposure: Misconfigured backup/data-transfer rules may expose app data.

Compromised / Rooted OS: Root privileges (UID 0) override sandbox isolation.

Debug Builds & Logs: `debuggable="true"` permits `run-as` extraction.

Access Control (Linux Sandbox)

Mechanism: Restricts file system read/write access at the OS layer via Linux UIDs (`0700` file mode).

Core Protection: *An ordinary malicious application cannot simply read another application's private SQLite database.*

Limitation: Protects who can open the file handle, but fails if secondary exposure vectors or root access exist.

Controls WHO can open the file handle at OS level

Encryption at Rest (SQLCipher / Keystore)

Mechanism: Cryptographically encrypts 4KB SQLite pages using AES-256 before writing to storage.

Key Protection: Master keys generated and managed inside hardware-backed Android Keystore (TEE/SE).

Nuanced Resilience: Encryption at rest provides an additional confidentiality layer if raw storage files become accessible, but a fully compromised runtime can still threaten protected data.

Controls WHAT can be read from raw storage bytes

RISK EVALUATION & IMPACT

Security Impact: The CIA Triad

Evaluating Systemic Harm in Mobile Client-Server Environments

CONFIDENTIALITY

HIGH IMPACT

Auth Token Theft: Extraction of plaintext JWT session tokens from unencrypted SQLite databases or SharedPreferences.

Private PII Exposure: Leakage of user chat messages, contacts, and personal records.

API Secret Leaks: Exfiltration of proprietary cloud backend API keys and sensitive parameters.

Potential Impact: Account Takeover & Data Breach

INTEGRITY

CRITICAL IMPACT

Confused Deputy Execution: Maliciously invoking internal administrative activities via Intent Redirection to alter settings.

Local Data Tampering: Modifying local SQLite database records via exported ContentProviders or backup injections.

Forged API Requests: Replaying or forging backend transactions using hijacked user credentials.

Potential Impact: Unauthorized State Modification

AVAILABILITY

MODERATE IMPACT

Intent Fuzzing Crashes: Sending malformed extras or unparseable parcels causing NullPointerException or BadParcelableException.

Background Service DoS: Flooding exported Services to trigger resource exhaustion and battery drain.

Database Lockout: Corrupting SQLite journal files, rendering the app unusable upon launch.

Potential Impact: Application Crash & Resource Exhaustion

* Note: Impact severity depends on exposed functionality, data sensitivity, and specific application context.

DEFENSIVE ENGINEERING

Defense Strategy: Multi-Layered Hardening

A Comprehensive Architectural Blueprint for Secure Android Apps

1. Manifest & Permissions

Explicit exported="false": Keep internal components non-exported by default.

Least Privilege: Request only strictly required permissions; audit `<uses-permission>` tags.

Permission Protection: Protect necessary exported endpoints with signature or custom permissions.

Backup Rules: Configure backup / data-extraction rules to exclude sensitive application data.

```
<activity
    android:name=".InternalUI"
    android:exported="false" />
```

2. Intent Security & IPC

Prefer Explicit Intents: Use explicit class bindings for all known internal targets.

Input Validation: Validate incoming actions, URIs, extras, and destination components.

Allowlist Destinations: Strictly check target class before forwarding nested Intents.

FLAG_IMMUTABLE: Enforce on PendingIntents by default to prevent hijacking.

```
// Verify destination before forwarding
if (allowedClasses.contains(target.className)) {
    startActivity(target)
}
```

3. Storage Security

Minimize Stored Secrets: Avoid persisting high-value credentials locally when possible.

App-Private Storage: Persist files exclusively in internal directories.

Android Keystore: Generate and protect cryptographic master keys in hardware TEE/SE.

Encrypt at Rest: Deploy SQLCipher page-level encryption where threat model requires it.

```
// Encrypt SQLite pages with SQLCipher
val db = SQLiteDatabase.openOrCreateDatabase(
    dbFile, passphrase, null)
```

PRACTICAL DEMONSTRATION PLAN

Final Demo: MITM Interception & Certificate Pinning

Securing the Client-Server Transit Channel in a Controlled Lab

Phase 1: Baseline Interception (Vulnerable)

Android Client → **Debug Network Config (User CA Trusted)** → Backend

Vulnerable / Test Configuration: Controlled debug Network Security Configuration explicitly trusts user-installed CAs (modern Android does not trust them by default).

Proxy Interception: Client traffic is routed through Burp Suite on port 8080; Burp issues dynamic TLS certificates.

Educational Lab Outcome: HTTPS traffic is decrypted in Burp proxy; plaintext credentials and chat tokens are inspected in transit.

Result: Plaintext credentials captured under vulnerable test configuration

Phase 2: Certificate Pinning Defense (Hardened)

Android Client → **[SHA-256 Pin Check] ✖ Proxy Rejected**

Application-Specific Trust Restriction: Client validates server's expected public-key SHA-256 hash using OkHttp `CertificatePinner` on top of normal TLS.

Pin Mismatch Detection: When Burp presents its interception certificate, the public key hash does not match the pinned value.

Connection Rejection: TLS connection is rejected immediately (`SSLPeerUnverifiedException`) before sensitive application payloads are exchanged.

Result: TLS connection rejected on pin mismatch • Handshake aborted

AUTHORIZED EDUCATIONAL LAB METHODOLOGY

1. Develop simple chat app → 2. Configure debug user-CA trust → 3. Intercept baseline credentials in Burp Suite → 4. Implement OkHttp CertificatePinner → 5. Verify TLS handshake rejection upon pin mismatch.

EXECUTIVE SUMMARY

Key Takeaways & Conclusion

Core Architectural Principles for Secure Mobile Engineering

1. Sandboxing Provides Application Isolation: Linux UID sandboxing isolates private files, but misconfigured component access (`android:exported="true"`) can expose internal interfaces if unprotected.

2. Untrusted IPC Requires Destination Validation: Crossing application boundaries via Intents creates a trust boundary. Untrusted IPC must be strictly validated before it reaches privileged functionality.

3. Access Control ≠ Encryption at Rest: Sandboxing controls who can access files; encryption protects what data reveals if files are extracted. Sensitive data requires appropriate access control and data protection.

4. Defense-in-Depth Across All Layers: Mobile security requires coordinated defenses across the Manifest, IPC message validation, local cryptographic storage, and network transit.

5. Theory Drives Practice: Controlled MITM interception and Certificate Pinning demonstrate how application-specific trust restrictions reject unauthorized interception before sensitive data is exchanged.

Thank you for your attention! We welcome any questions from the instructor and class.

ACADEMIC SOURCES

References & Industry Standards

Authoritative Literature, Frameworks & Specifications

[1] Android Developers

"Permissions on Android"

Android Developers. [Online]. Available: https://developer.android.com/guide/topics/permissions/overview. [Accessed: Oct. 1, 2026].

Protection Levels & Runtime Governance

[2] Android Developers

"Intents and Intent Filters"

Android Developers. [Online]. Available: https://developer.android.com/guide/components/intents-filters. [Accessed: Oct. 1, 2026].

Component Resolution & IPC Architecture

[3] Android Developers

"Remediating Intent Redirection Vulnerability"

Android Developers. [Online]. Available: https://developer.android.com/privacy-and-security/risks/intent-redirection. [Accessed: Oct. 1, 2026].

Vulnerability Mechanics & Remediation Guidance

[4] OWASP Foundation

"Mobile Application Security Verification Standard (MASVS v2.0)"

OWASP Foundation. [Online]. Available: https://mas.owasp.org/. [Accessed: Oct. 1, 2026].

MASVS-STORAGE & MASVS-IPC Verification Standard

[5] OWASP Foundation

"Mobile Application Security Testing Guide (MASTG)"

OWASP Foundation. [Online]. Available: https://mas.owasp.org/. [Accessed: Oct. 1, 2026].

Technical Testing Procedures for Mobile Storage & IPC

[6] NIST

"Vetting the Security of Mobile Applications"

NIST Special Publication 800-163 Rev. 1, Jul. 2019. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-163r1.

Federal Guidelines for Mobile Application Vetting